1. Product scope and feature validation

StewardLab

Evolution-Aware Antibiotic Strategy Evaluation
Final Technical Product Report | BSc Software Engineering

Anjeline Noel Odero | Supervisor: Dirac Murairi | 24 October 2026
Repository baseline: 42fdd0e | https://github.com/noel-odero/Capstone-101

StewardLab integrates reinforcement learning with an interactive research workbench for exploring antibiotic sequences in a simplified E. coli resistance environment. Users record labeled samples, compare frozen PPO and DQN policies with a random susceptible-action baseline, and inspect stored trajectories and evidence. The product is locally operational. It does not diagnose, prescribe or predict patient outcomes.

Alignment with the approved proposal [1]

Planned feature	Intended purpose	Status	Implementation evidence / qualification
Resistance environment	Represent sequential changes	Completed*	simulation/: seven resistance bits, actions, stochastic transitions and reward.
Trained PPO + baseline	Compare sequential strategies	Completed	ml/ and artifacts/frozen/public-data-v2/; DQN is an additional comparator.
Susceptibility input	Define the starting profile	Completed	frontend/src/features/simulation/ and samples/: validated switches and saved labels.
Prior treatment-history input	Represent previous exposure	Not implemented	Generated action histories are saved; users cannot supply pre-analysis treatment history.
Scenario configuration	Control evaluation conditions	Partial	Profiles and seeds are configurable. App horizon and scenario are fixed by the verified model.
Outcomes and decision traces	Inspect consequences and support	Completed*	recommendation/ResultsWorkbench.tsx: sequences, metrics, trajectories and CS/XR evidence.
Persistence and model metadata	Revisit and reproduce runs	Completed	PostgreSQL records owner-scoped inputs, seeds, policy identities, results and step traces.
Seed/scenario evaluation	Assess robustness	Partial	20 training seeds, separate evaluation draws, exact reference and occurrence sensitivity; no held-out profile split.
Regional starting conditions	Reflect local evidence	Not calibrated	Exhaustive binary profiles are evaluated; no Rwanda-specific prevalence distribution is fitted.
Practitioner validation	Assess usability and clarity	Pending	The planned 3-5 practitioner study has not yet been conducted.

***Scope refinement.** Resistance burden replaces a direct clinical-success objective; susceptibility compliance is not efficacy. Evidence is incomplete and probabilities are scenario assumptions. The original urinary-tract 3D page is illustrative only. These refinements narrow the claims made about the delivered model.

Extensions: DQN, a labeled-sample queue, separate Analysis History and Insights pages, research collections, exports and Model Explorer strengthen research continuity. They do not replace the unimplemented prior-history input. Proposal milestones are evidenced by implementation and experiments, not independently verified against a dated milestone log.

2. Architecture, data handling and code quality

The implemented stack follows the proposal: React/TypeScript, ASP.NET Core, PostgreSQL/EF Core and a Python FastAPI service using Gymnasium and Stable-Baselines3. Scientific computation is separate from application ownership and persistence.

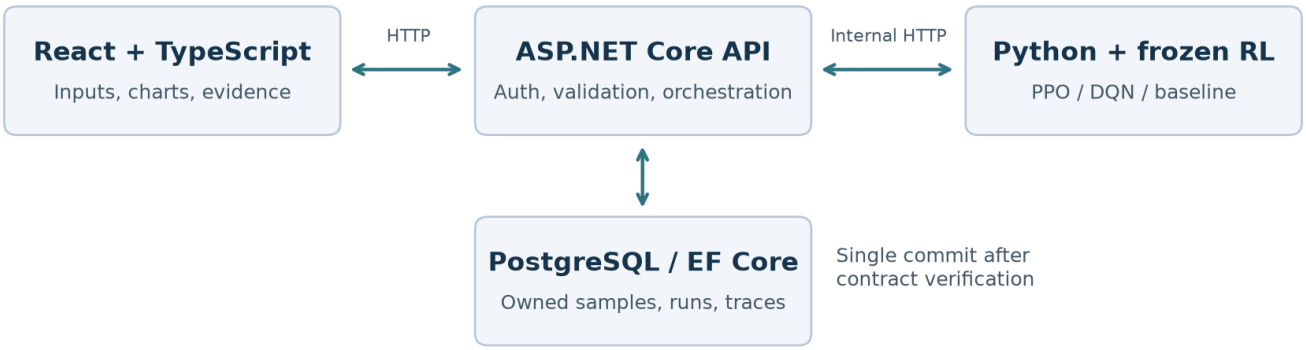


Figure 1. A comparison request uses the API as the public boundary; Python performs inference and the API stores verified outputs.

Request-to-result workflow

1. The researcher signs in and chooses a saved or newly configured profile. React submits the profile and optional seeds with a cookie session and CSRF token.
2. The API validates shape, ranges and sample ownership, then sends one request to Python. Both learned checkpoints and their data were hash-verified at service startup.
3. Python executes the three policies from shared initial conditions and checks the pairing contract. Seeds match by step, but different actions can yield different outcomes.
4. The API validates the returned contract and commits the run, strategy records and traces in one transaction. Only then does it return 201 Created with the resource ID and isPersisted=true.

OOP and maintainability practices

Practice	Concrete implementation
Encapsulation	Immutable ResistanceState and validated inference context protect scientific invariants.
Abstraction / polymorphism	IRunRepository and IPythonServiceClient isolate infrastructure; ObservationPolicy supplies a common action-selection contract.
Inheritance	MaskedDQN extends library DQN; the environment implements Gymnasium behaviour; React has a scene error boundary.
Separation of concerns	Domain, Application, Infrastructure and API projects; frontend modules group samples, evaluations, results, history and exploration.
Dependency inversion / reuse	Constructor-injected interfaces, shared DTO mappings, metrics, validators, charts and UI components. Architecture tests enforce dependency directions.

Persistent data and failure handling

Users, samples, model_versions, experiment_runs, strategy_runs, step_traces and analysis_collections form the stored research record. Reads are account-scoped; missing and foreign-owned records share a 404 response. Transactions prevent partial saved comparisons. Seeds, checkpoint identities, data hashes and metric definitions are retained rather than recomputed when a run is reopened.

Passwords use the framework salted PBKDF2 hasher. Cookie sessions, antiforgery checks and authentication rate limiting are implemented. Validation failures return 422; unavailable dependencies and invalid upstream contracts are classified explicitly. Correlation IDs support diagnosis, and scientific POSTs are not automatically retried.

Code evidence: backend/src/Amr.Application/Services/ComparisonService.cs; backend/src/Amr.Infrastructure/Persistence/RunRepository.cs; backend/tests/Amr.UnitTests/Architecture/; ml/src/evaluation_runner.py. Source references resolve within repository baseline [2].

3. Algorithms and model evaluation

Scientific representation and custom logic

The canonical action order is CIP, NIT, FOS, TMP, GEN, MEC and CAZ. A binary profile uses 0=susceptible and 1=resistant, giving 128 possible states rather than 128 measured isolates. Policy observations contain seven resistance bits, seven previous-action indicators and the step: 15 values. A hard susceptibility mask constrains PPO prediction and DQN exploration, prediction and Bellman-target computation.

The training table contains 30 directed records across 18 pairs; 15 unknown-direction records remain in the evidence registry and are excluded from training. Applicable XR, CS and neutral candidates are deduplicated by target/outcome and sampled uniformly in REF_UNIFORM_SUPPORTED. Empty candidate sets leave the state unchanged and are recorded as unsupported. Transition probabilities are assumptions, not measured clinical probabilities [3,4].

Reward is minus the post-action resistant count divided by seven, with an all-resistant terminal cost carried through the remaining horizon. Horizon=8 and discount=1. Burden is negative return. The baseline samples susceptible actions uniformly. A finite-horizon, memoized dynamic program calculates an exact reference conditional on the same transition model.

Twenty-replicate comparison

Each PPO and DQN replicate used 100,096 environment interactions; update schedules and compute differ. Training covers all 127 nonterminal profiles; evaluation covers all 128 across five separate RNG blocks. Results are averaged within each training replicate, then across 20 replicates. Evaluation holds out random draws, not profiles. Reused baseline draws are not independent extra baseline replicates [3].

Strategy	Burden (lower)	Emergence	Options (higher)	Unsupported steps
PPO	2.8885	0.5098	4.8802	2.8633
DQN	2.8835	0.5234	4.8816	2.3258
Random baseline	3.7768	1.1516	3.7484	1.6266

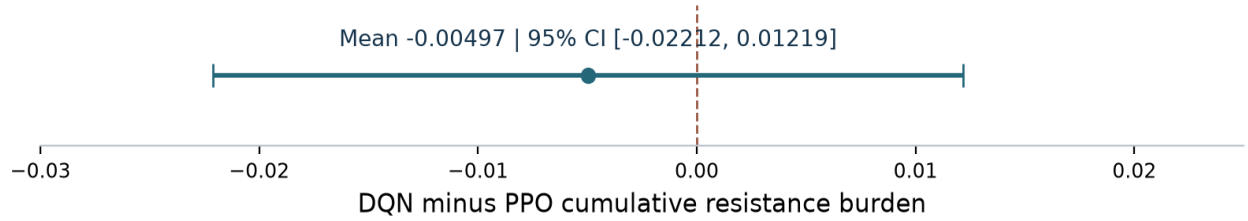


Figure 2. Paired mean burden difference across 20 training replicates. The interval includes zero; negative values favor DQN.

DQN minus PPO mean burden was -0.00497, with a 95% Student-t interval of [-0.02212, 0.01219]. Ten seeds favored each policy. The analyzer flags strong skew, so this interval is approximate. These results do not establish algorithm superiority or equivalence. Both have lower recorded mean burden than this baseline within the assumed environment.

All strategies have exposure 7.9375 and susceptible-action rate 1.0 in the aggregate: the terminal profile has zero actions while other episodes reach eight. These are contextual/mask checks, not evidence of clinical success. The app freezes PPO seed 19000 and DQN seed 15000 after fresh validation and separate confirmation draws. Selection identifies deployment checkpoints, not an overall algorithm winner.

4. Testing, validation and performance

Verification combines component-level tests, cross-service integration and model-specific audits. Counts below are the latest completed local checks on 24 October 2026, not a claim about a hosted CI run or code-coverage percentage [3].

Approach / suite	Passed	Purpose and representative coverage
Python	499	State/observation encoding, rewards, transitions, masks, training, inference and data integrity.
Frontend component / functional	124	Profile input, navigation, authentication, result schemas, samples, Insights and exports; mocked HTTP.
.NET unit	281	Application logic, mappings, contracts, validation, authentication and architecture rules.
.NET integration	274	Controllers, PostgreSQL ownership, migrations, schema constraints and saved-run retrieval.
Live integration	10	Real Python service through the .NET API layer with real PostgreSQL; distinct from a browser automation test.
Total	1,188	All passed; frontend build/lint, .NET warning-as-error build, data check and 13/13 frozen checksums also passed.

Why these approaches were selected

Unit/model tests isolate scientific and validation rules that visual inspection could miss: invalid action masks, terminal-tail costs and unsupported transitions. **Integration/live tests** test serialization, real migrations, account isolation and durable results across service boundaries. **Frontend tests** exercise user workflows and malformed/missing responses; they use jsdom and do not prove real-browser GPU behaviour.

Improvements after failures: migration tests were updated after the collections schema was added; obsolete playback assertions were replaced with current 3D-route and three-strategy checks. A running Vite instance retained stale import resolution after reorganization; restarting it restored served modules. The incident showed why API health alone does not establish frontend readiness.

Different-input checks against the running Python service

Input	Observed response
0000000; 1000001; 1111110	HTTP 200; all three strategies return an eight-action trajectory.
1111111	HTTP 200 terminal result; zero actions and burden 8 for each strategy.
Six values; a value of 2	HTTP 422: request_validation_error; invalid_context respectively.

Measured performance and compatibility limits

On Windows 11 Enterprise (10.0.26200), AMD Ryzen 5 PRO 8540U and approximately 32 GB RAM, three warm requests per profile gave median internal-comparison times of 30.18 ms (0000000), 28.10 ms (1000001), 29.64 ms (1111110) and 15.85 ms (terminal). These non-persisting requests exclude browser rendering, public-API persistence, cold starts and concurrent load.

Local Windows execution is verified. CI defines Windows Python tests and Ubuntu .NET/live jobs; current remote results were not verified in this review. Responsive layouts, lazy scene loading, instanced particles, bounded pixel ratio and WebGL fallbacks are implemented. Cross-browser/version testing, low/high-end device comparison, 3D FPS, browser memory and public deployment performance remain unmeasured.

5. Limitations, reproducibility and delivery

Interpretation and outstanding validation

This prototype demonstrates an integrated computational research workflow. Clinical effectiveness, patient-specific treatment advice and real bacterial-population evolution are outside the implemented model. The urinary-tract 3D animation is aesthetic; measured model outputs are the resistance trajectories and recorded action traces.

Limitation	Consequence and next step
Incomplete transition coverage	64 of 448 feasible state/action pairs lack applicable transitions; 114 force an unchanged state including neutral outcomes. Make gaps visible; expand evidence before adding biological claims.
Restricted evolutionary dynamics	There is no selected-drug self-resistance, population size, dose, recovery or stopping model. Repeated selections and zero burden can reflect these assumptions.
Limited generalization claim	Profiles are known during training. Occurrence sensitivity varies assumptions; it does not establish unseen-profile, regional or clinical generalization.
Incomplete original scope	Prior treatment-history input and regional starting-distribution calibration are absent. Practitioner usability testing has not yet occurred.
Unfinished acceptance evidence	A browser/device performance matrix and public-hosting evidence remain outstanding. The Privacy page has a stale Google-account description requiring correction.

The planned usability exercise should recruit 3-5 relevant practitioners with consent, use common input/interpretation tasks, and record task completion, misunderstandings and structured feedback. No usability scores or participant outcomes are reported because that study has not been conducted.

Reproduce and hand over the implementation

README.md documents Python 3.11, Node >=22.12, the pinned .NET 10 SDK, Docker/PostgreSQL, dependency installation and explicit EF migrations. Start PostgreSQL, Python (8000), API (5080) and Vite (5173). Models load from local files. docs/api.md covers tests; docs/scientific-model.md defines semantics; docs/research.md documents data integration, benchmarking, selection and exact review.

The frozen release retains checkpoints, configurations, shared data, selection records and SHA256SUMS.txt; ml/third-party-notices.txt retains DQN attribution. submission/verification-evidence.json preserves study summaries, paired per-seed differences, source hashes and the local verification record. Full generated study artifacts remain excluded from Git and can be reproduced with the documented module commands.

Submission packaging: link this PDF and the uploaded walkthrough from README.md. The second-attempt ZIP must match the submitted repository and include the actual report and recorded video. This report and the accompanying script do not constitute the recorded video. No public deployment or completed practitioner study is claimed.

References and evidence

[1] Odero, A. N. (2026). An AI-Based Decision Support System for Antibiotic Stewardship in Multidrug-Resistant Escherichia coli Urinary Tract Infections. Approved proposal supplied for scope comparison; especially sections 1.5 and 3.4-3.14.
[2] StewardLab source repository, baseline 42fdd0e: https://github.com/noel-odero/Capstone-101/tree/42fdd0ec968c5600abb71140e22c839a8101331a.
[3] submission/verification-evidence.json; experiments/runs/public-data-v2/report.json; experiments/results/public-data-v2-review-rerun/analysis.json; artifacts/frozen/public-data-v2/selection.json.
[4] data/research/transition-data-2026-10-23/sources.json and data/processed/evidence_registry.csv: public experimental extracts, source URLs and hashes. See docs/research.md for derivation and limitations.